ML4N - LAB

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Matplotlib **3**
1.1 Introduction to Matplotlib 3
1.1.1 Two Interfaces: Stateful vs. Stateless 3
1.2 Stateless: Figures and Axes 4
1.2.1 Creating a Figure and Line Plot 4
1.2.2 Creating Many Subplots 5
1.3 Plot Types 6
1.3.1 Line and Scatter Plots 6
1.3.2 Bar Charts 7
1.3.3 Histograms and Text Annotations 8
1.3.4 Box Plots 9
1.3.5 pandas Plotting Integration and Saving 10

Chapter 1

Matplotlib

1.1 Introduction to Matplotlib

Matplotlib is the foundational plotting library in Python. It is used extensively on its own, and serves as the underlying drawing engine for pandas and Seaborn. The standard import convention is `import matplotlib.pyplot as plt`.

1.1.1 Two Interfaces: Stateful vs. Stateless

Matplotlib can be used in two distinct ways[cite: 156]:

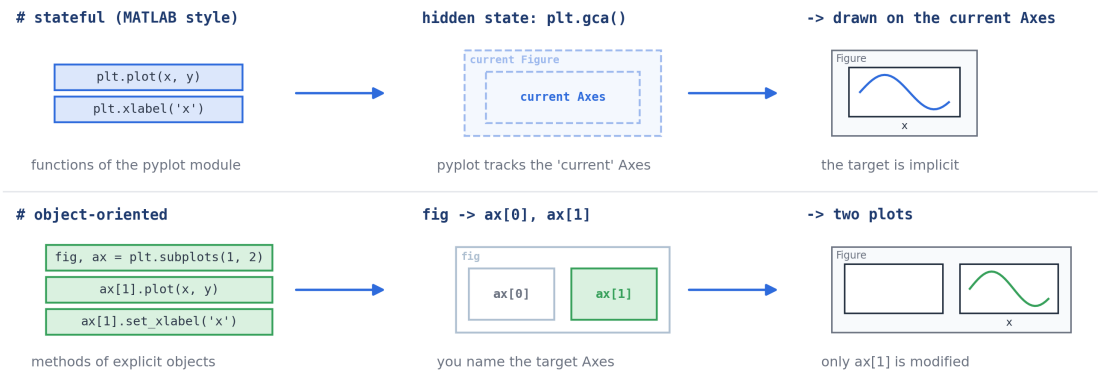


Figure 1.1: The two Matplotlib interfaces: `pyplot` functions act on the current Axes, tracked implicitly by Matplotlib, while the object-oriented style calls methods on explicit `Figure` and `Axes` objects, so the target of each call is always clear.

- **Stateful (MATLAB style):** You call functions directly from the `pyplot` module (e.g., `plt.plot()`, `plt.xlabel()`). These functions automatically apply changes to the *current* active Figure and Axes, which Matplotlib tracks in the background.
- **Stateless (Object-Oriented):** You explicitly call methods on specific `Figure` and `Axes` objects (e.g., `ax.plot()`, `ax.set_xlabel()`). This approach is strongly recommended because it makes it perfectly clear which plot you are modifying, especially when working with multiple plots at once.

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-10, 10, 100)
y = x ** 2

# --- Stateful (pyplot) Interface ---
plt.figure(figsize=(5, 3))
plt.plot(x, y)          # Draws on the "current" figure
plt.ylabel('Y axis')
plt.show()              # Displays the figure

# --- Stateless (Object-Oriented) Interface ---
fig, ax = plt.subplots(figsize=(5, 3))
ax.plot(x, y)           # Draws explicitly on 'ax'
ax.set_ylabel('Y axis')
plt.show()
```

Listing 1.1: Stateful vs. Stateless (Object-Oriented) approaches in Matplotlib.

1.2 Stateless: Figures and Axes

A **Figure** is the overall window or image, whereas the **Axes** are the individual plots contained inside the Figure, each with its own x-axis and y-axis. A single Figure can hold multiple Axes.

1.2.1 Creating a Figure and Line Plot

- `plt.subplots(figsize=(width, height))` creates and returns a new Figure and its associated Axes. The dimensions are specified in inches.
- `ax.plot(x, y)` takes two lists or NumPy arrays containing coordinates and joins the points with line segments. Calling it multiple times on the same Axes automatically adds new lines in different colors.
- You can configure the plot's appearance using `ax.set_...` methods (e.g., `ax.set_xlabel`, `ax.set_xlim`, `ax.set_xticks`) and add a grid with `ax.grid()`.

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(5, 3))

# Plotting two lines on the same Axes
ax.plot([0, 1, 2], [2, 4, 6])
ax.plot([0, 1, 2], [3, 6, 9])

# Setting attributes explicitly
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_xlim([0, 2])
ax.set_xticks([0, 1, 2])

```

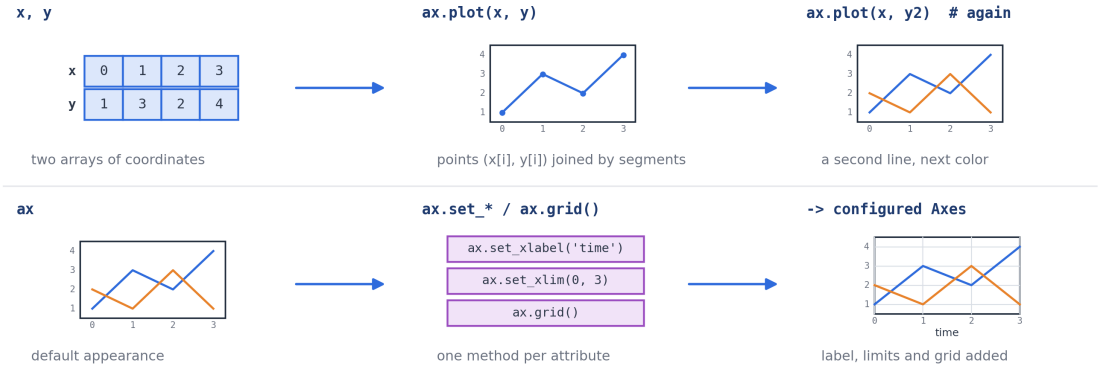


Figure 1.2: Line plots: `ax.plot(x, y)` joins the points `(x[i], y[i])` with segments, and each new call adds a line in the next color. The `ax.set_...` methods and `ax.grid()` change one attribute of the Axes at a time.

```
# Adding a grid
ax.grid()

plt.show()
```

Listing 1.2: Creating a plot and setting axis attributes using the stateless interface.

1.2.2 Creating Many Subplots

You can create a grid of plots by passing the number of rows and columns as the first two arguments to `plt.subplots()`.

- **1D Grid:** If you create one row of n Axes, the returned `ax` object is a 1-D NumPy array of shape `(n,)`. You access the second plot using `ax[1]`.
- **2D Grid:** If you create a grid of m rows and n columns, `ax` is a 2-D array of shape `(m, n)`. You access the bottom-right plot using coordinate indexing, like `ax[1, 1]`.
- **Tight Layout:** Always call `fig.tight_layout()` at the end of your plotting code to automatically adjust spacing and prevent the labels of neighboring subplots from overlapping.

```
import matplotlib.pyplot as plt

# --- 1D Grid: 1 Row, 2 Columns ---
fig1, ax1 = plt.subplots(1, 2, figsize=(5, 2.5))

# ax1 is a 1-D array of shape (2,)
ax1[0].plot([0, 1, 2], [2, 4, 6])  # First plot
ax1[1].plot([0, 1, 2], [3, 6, 9])  # Second plot

fig1.tight_layout()
plt.show()

```

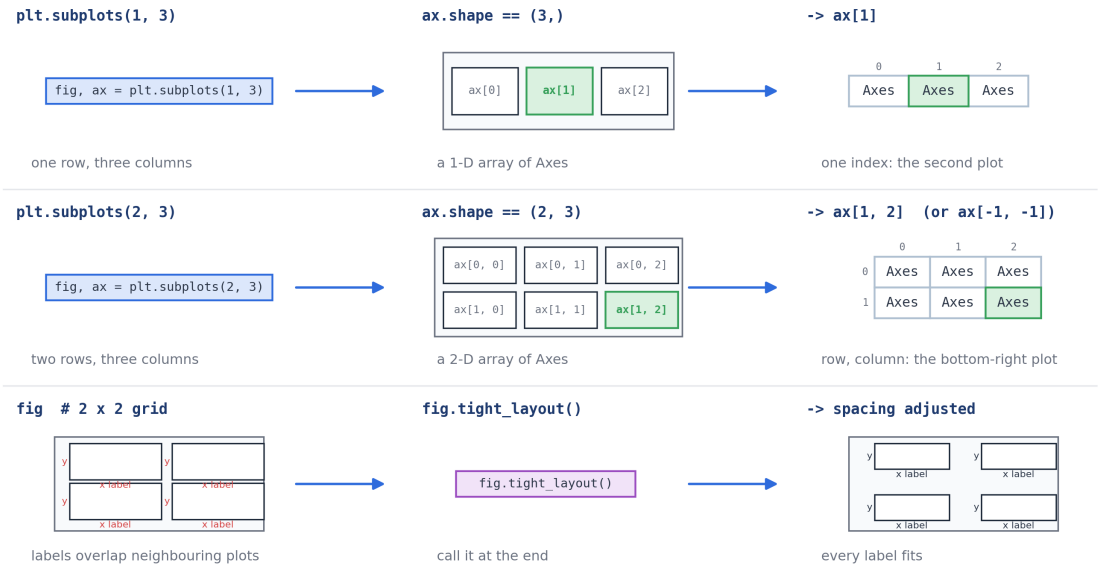


Figure 1.3: Subplots: with one row, `ax` is a 1-D array indexed as `ax[j]`; with several rows and columns it is a 2-D array indexed as `ax[i, j]`. `fig.tight_layout()` adjusts the spacing so that labels do not overlap.

```
# --- 2D Grid: 2 Rows, 2 Columns ---
fig2, ax2 = plt.subplots(2, 2, figsize=(5, 4))

# ax2 is a 2-D array of shape (2, 2)
ax2[0, 0].plot([0, 1, 2], [2, 4, 6])  # Top-left
ax2[1, 1].plot([0, 1, 2], [3, 6, 9])  # Bottom-right

fig2.tight_layout()
plt.show()
```

Listing 1.3: Creating and managing 1D and 2D grids of subplots.

1.3 Plot Types

Matplotlib provides various methods to visualize data. All of these methods are called directly on the `Axes` object (`ax`).

1.3.1 Line and Scatter Plots

- **Line Plot (`ax.plot`):** Draws a sequence of points joined by segments, making it ideal for functions or time series[cite: 163, 171]. You can customize it using keyword arguments like `linestyle` (`'-'`, `'--'`, `':'`), `marker` (`'o'`, `'*'`), `color` (or `c`), and `label` (used to generate a legend).

- **Scatter Plot (`ax.scatter`):** Draws a cloud of independent points, useful for showing the relationship between two features. Unlike line plots, you can assign an individual color (`c`) and size/area (`s`) to every single point. Colors are usually mapped to numerical values using a colormap (`cmap`).

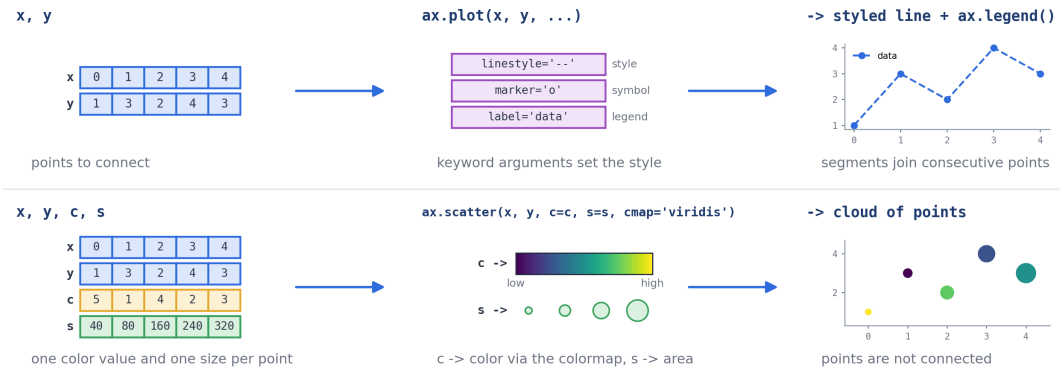


Figure 1.4: Line and scatter plots: `ax.plot` joins consecutive points and is styled with keyword arguments such as `linestyle`, `marker` and `label`. `ax.scatter` draws unconnected points, each with its own color (`c`, mapped through `cmap`) and area (`s`).

```python
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(5, 3))

# --- Line Plot ---
x = np.linspace(0, 5, 20)
y = np.exp(x)

# Customizing line style, color, and marker
ax.plot(x, y, c="blue", linestyle="none", marker="*", label="Points only")
ax.plot(x, 2*y, c="green", linestyle="--", label="Dashed curve")
ax.legend(loc="upper left")

# --- Scatter Plot ---
fig2, ax2 = plt.subplots(figsize=(5, 3))
x_scat, y_scat = np.random.rand(50), np.random.rand(50)

# Colors and sizes defined per point
colors = x_scat + y_scat
area = 100 * (x_scat + y_scat)

points = ax2.scatter(x_scat, y_scat, c=colors, s=area, cmap="viridis")
fig2.colorbar(points, ax=ax2)  # Adds a legend for the colormap
```

Listing 1.4: Creating line plots and scatter plots with varying colors and sizes.

1.3.2 Bar Charts

- **Bar Chart (`ax.bar`):** Displays one numerical value per category.
- To group multiple series of bars side-by-side, you can manually shift their positions along the x-axis (e.g., by subtracting or adding half the bar `width`) and update the ticks to show the category names.

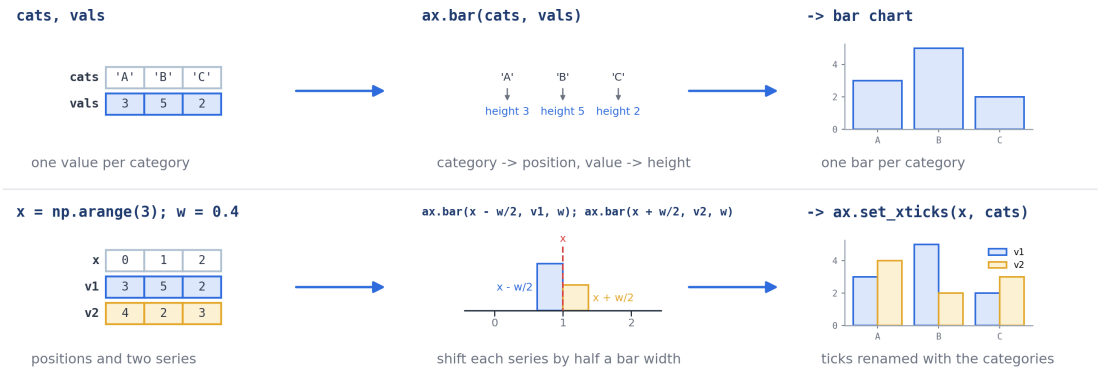


Figure 1.5: Bar charts: `ax.bar` draws one bar per category. To group two series, each is shifted by half a bar width around the integer positions `x`, and `ax.set_xticks(x, cats)` puts the category names back on the axis.

```python
import matplotlib.pyplot as plt
import numpy as np

height_min = [10, 2, 8]
height_max = [8, 6, 5]
labels = ["Sensor 1", "Sensor 2", "Sensor 3"]

x = np.arange(3)  # [0, 1, 2]
width = 0.4

fig, ax = plt.subplots(figsize=(5, 3))

# Shifting bars to the left and right to group them
ax.bar(x - width / 2, height_min, width=width, label="min")
ax.bar(x + width / 2, height_max, width=width, label="max")

# Setting the custom text labels on the x-axis
ax.set_xticks(x, labels)
ax.legend()
```

Listing 1.5: Creating grouped bar charts.

1.3.3 Histograms and Text Annotations

- **Histogram (`ax.hist`):** Shows the frequency distribution of a single variable, counting how many values fall into consecutive intervals (bins). The `bins` parameter can be an integer (to specify the number of equal bins) or a NumPy array defining the exact bin edges.

- **Annotations (`ax.text`):** You can write text directly on the plot by passing data coordinates (`x`, `y`). Matplotlib can render LaTeX math equations if the string is enclosed in dollar signs (e.g., `$\mu$`).

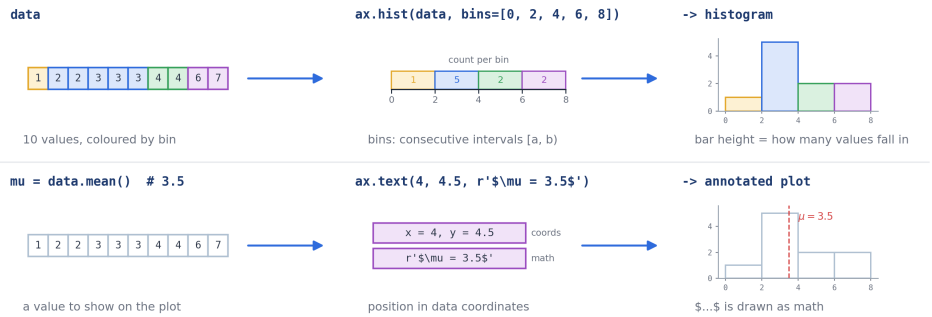


Figure 1.6: Histograms and annotations: `ax.hist` counts how many values fall into each bin (here with explicit edges), and `ax.text` writes a string at a position given in data coordinates, rendering the part between dollar signs as math.

1.3.4 Box Plots

- **Box Plot (`ax.boxplot`):** Used to summarize and compare several distributions side-by-side.
- The central box extends from the first quartile (Q1) to the third quartile (Q3), with a line marking the median.
- "Whiskers" extend outward to the farthest data points that fall within 1.5 times the Inter-Quartile Range (IQR). Any points beyond the whiskers are plotted as individual markers and are considered candidate outliers (fliers).

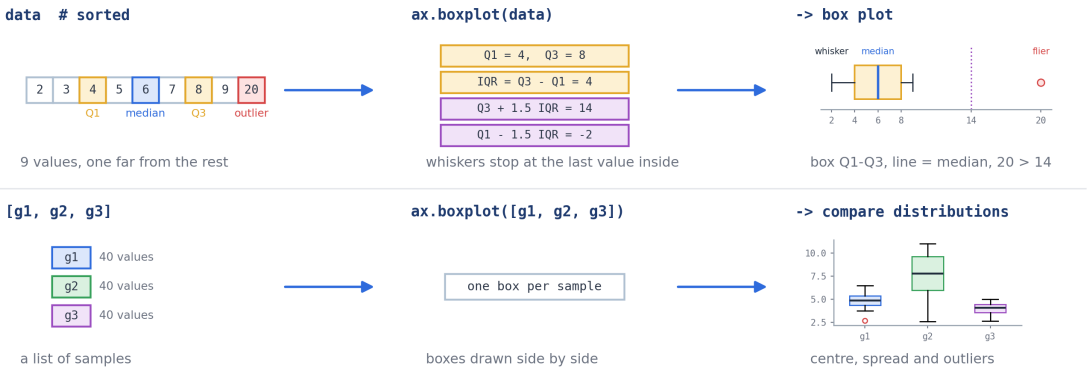


Figure 1.7: Box plots: the box spans Q1 to Q3 with a line at the median; the whiskers reach the farthest values within 1.5 IQR of the box, and values beyond are drawn as fliers. Passing a list of samples draws one box per sample for comparison.

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

fig, (ax_hist, ax_box) = plt.subplots(1, 2, figsize=(10, 4))

# --- Histogram ---
heights = np.random.normal(170, 10, 500)
```

```python
bins = np.arange(140, 200, 2.5)  # Defining exact bin edges

ax_hist.hist(heights, bins=bins)
# Annotating text at specific data coordinates, using LaTeX formatting
ax_hist.text(142, 40, r"$\mu = 170$", fontsize=10)
ax_hist.set_title("Height distribution")

# --- Box Plot ---
short = np.random.normal(170, 10, 500)
tall = np.random.normal(190, 10, 500)

ax_box.boxplot([short, tall])
ax_box.set_xticks([1, 2], ["Short", "Tall"])
ax_box.set_title("Box Plot Comparison")

fig.tight_layout()

# --- pandas Plotting & Saving ---
# pandas can draw directly onto existing Matplotlib Axes
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]}, index=["row1", "row2"])
df.plot.bar(ax=ax_box)  # Overlays a bar chart on ax_box (example)

# Save the figure to disk
fig.savefig("my_plots.pdf")
plt.close(fig)  # Closes the figure to free memory
```

Listing 1.6: Plotting histograms with text annotations and box plots.

1.3.5 pandas Plotting Integration and Saving

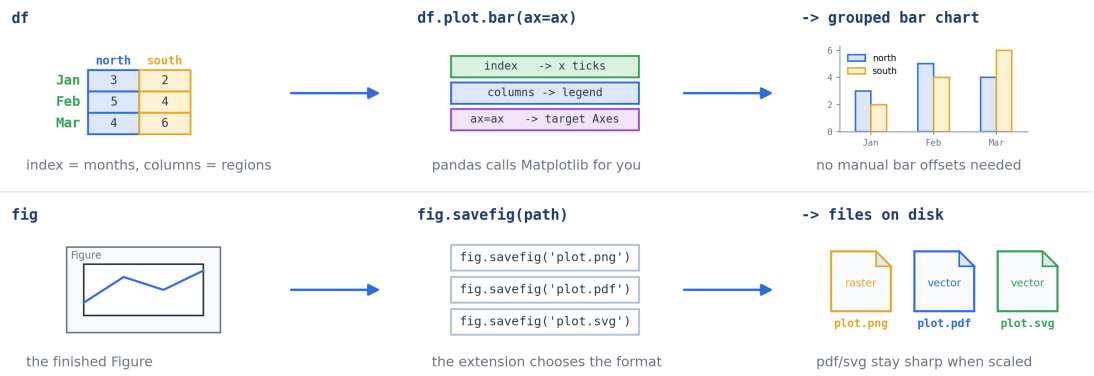


Figure 1.8: pandas plotting and saving: `df.plot.bar(ax=ax)` uses the index for the x-axis ticks and the column names for the legend, drawing on the given Axes. `fig.savefig` chooses the file format from the extension: PNG is a raster format, PDF and SVG are vector formats.

pandas Series and DataFrames have a built-in `.plot` accessor (e.g., `df.plot.bar()`) that automatically generates Matplotlib charts. It conveniently uses the DataFrame's index for the x-axis and its column names for the legend. By passing an `ax` argument (`ax=ax`), you can direct pandas to draw on your specific stateless Axes.

To save your final figure to disk, use `fig.savefig(path)`. Matplotlib automatically determines the file format based on the extension you provide (`.png`, `.pdf`, `.svg`, `.jpg`).